When Does Admission Control Pay? Decomposing End-to-End Wall Clock in Memory-Bound Agentic LLM Serving

Anonymous Submission

***Abstract*—Agentic LLM workloads differ from chat in a way that makes admission control look like the obvious lever: each session is a long, multi-round job whose KV footprint grows every round, and a single session can occupy most of a memory-constrained GPU's KV pool. A natural hypothesis follows—a controller that reads engine telemetry and admits on measured capacity should beat a static concurrency cap. We test that hypothesis directly and find that, on our workload and hardware, it does not hold: across nine controller variants and a reimplementation of an independent congestion-based controller, none reaches the static Pareto front at the strict latency threshold interactive serving cares about, and a plain cap on resident sessions remains the fastest policy we measured. We report how far that verdict is conditional—at a loose threshold one variant crosses the front by a margin we cannot resolve against run-to-run noise—and we treat the conditional version as the claim.**

We give a decomposition that locates the reason, and the decomposition is the contribution. We split the end-to-end wall clock into *busy* and *idle* time, and show that the busy component is governed by two quantities the admission decision does not control: the per-step cost and the number of requests actually in flight. Admission policy moves the second—but the first rises with it, and takes back about a third of what the extra concurrency buys. On configurations without a host cache tier we can account for that rise token for token: evicted prefixes are recomputed at close to one for one, and the recomputation is charged to the decode step. The idle component (7.4–36.3% of the wall clock) is structural: an agent session issues no request while waiting on a tool, and we measure that this wait averages 16.85 s, not the sub-second figure a naive reading of the trace suggests. We further show that the same throttle setting that is worth 51% at low load costs 42% at high load—so no single scalar can be optimal across regimes, which is precisely the regime in which adaptive control could have paid. We report four mechanistically distinct policy routes we falsified, each with its measured reason, and a reimplementation of an independent congestion-based controller that also lands below the front. The constructive output is a single measurable number: the elasticity of per-step cost with respect to the running count. On configurations where the mechanism is confirmed token for token it is 0.6, meaning every 10% of added concurrency buys about 4% of throughput—which is why no adaptive policy we built could move the frontier. We argue that prior systems' reported gains from state-preserving suspension are largely already realised by the permit-per-session design our controller inherits, and we state the boundary of that argument.

***Index Terms*—LLM serving, admission control, KV cache, agentic workloads, performance characterization, negative results**

I. Introduction

An agentic LLM session is not a request. It is a long-lived job that issues a round of inference, waits on a tool, and comes back—tens of times—with a context that grows every round. On a memory-constrained accelerator this makes admission control look like the obvious lever. The KV pool is the binding resource; a single large session can occupy most of it; and the sessions themselves are wildly heterogeneous. If a scheduler can see how much of the pool a candidate will need and how much room is left, surely it should beat a fixed number.

We believed this, built a controller that does exactly that, and spent a considerable amount of GPU time discovering that on our workload and hardware it is false. This paper reports the negative result, but its contribution is not the negative result—it is the decomposition that explains it, and the boundary that decomposition draws.

A. What we measured

We serve a replay of 200 real coding-agent sessions on a single RTX 3090 (Qwen2.5-Coder-7B, 96K context, SGLang, a 101,432-token KV pool) and compare a family of admission policies against a sweep of static concurrency caps. Nine controller variants, spanning four mechanistically distinct ideas, all land on or below the static Pareto front (Fig. 1). The best of them reaches the front and no further.

B. Why

We decompose the wall clock into *busy* time, during which the engine has at least one request in flight, and *idle* time, during which it has none. The busy component is governed by $W_{\text{busy}} \approx (G/\overline{R})\,\tau$ (Eq. 3), where $\overline{R}$ is the mean number of running requests, G the generated tokens and τ the per-step cost. An admission policy controls $\overline{R}$ and nothing else. The difficulty is that τ is not constant in $\overline{R}$: it rises from 27.0 to 36.7 ms as the busy-time running count goes from 1.00 to 1.84, and that rise absorbs a third of what the extra concurrency buys.

The idle component is structural rather than a control failure. A session in a tool wait issues no request, so when all live sessions happen to be waiting, the engine is genuinely empty—and we measure that the median zero-occupancy stretch is 20.6 s and 69.6% of all idle *time* sits in stretches over 20 s, an order of magnitude above the 2 s control cycle. Crucially, the tool wait itself is much longer than the trace's median suggests: the median inter-round gap is 0.35 s, but the *mean* is 16.85 s with 22% of gaps over ten seconds. A controller that reads only the median concludes, as we initially did, that

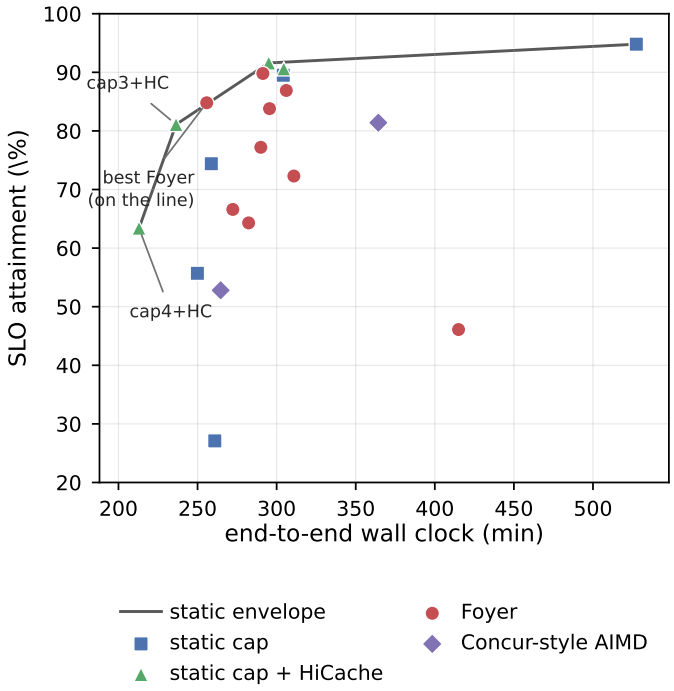


Fig. 1. Every completed run on the (wall clock, SLO) plane. The grey line is the upper-left envelope of the static points; a policy below or on it has not beaten a static cap, because time-slicing two caps approximates any point on the segment between them. Nine controller variants, none above.

sessions never leave and therefore that there is no idle time to recover. There is.

C. Contributions

- We falsify, with pre-registered criteria, four mechanistically distinct routes by which an adaptive admission controller was expected to beat a tuned static cap on this workload: changing the accounting base, changing the growth reserve, changing the hard ceiling, and changing the execution granularity from session-count to running-request-count. Each is reported with its measured reason (§IV).
- We give a two-part decomposition of the wall clock (§III) that locates the cancellation: admission policy moves the running-request count, and the per-step cost moves with it. We also report the mechanism we proposed for that rise and had to withdraw, with the two engine counters that refute it (§III); the negative result does not rest on it, because the cancellation is measured rather than inferred.
- We reduce the question to one measurable number, the elasticity of per-step cost with respect to the running count (§VII), and measure it at 0.38 on the model-consistent step cost and 0.61 on the per-request one, on the configurations whose mechanism we can account for token for token. This is the paper's constructive output: it decides in minutes, from logs a serving system already writes, whether admission control has room.
- We characterize when adaptivity *could* have paid, and show that our workload is not that case: the same throttle configuration that is worth $+51\%$ at 25 sessions costs -42% at 200 (§V). Put differently, no single scalar cap is optimal across both, which is the regime adaptive control exists for—and we measure that adaptive control does not reach it either.
- We make the measurements reproducible: the verdict of every run is produced by a single script from the raw per-step logs, and we report the six claims we had to retract along the way, with the checks that caught them.

D. What we do not claim

We do not claim that admission control is useless in general. We claim it does not pay *here*, we show the measurement that decides it, and we argue (§VII) that the same measurement can be taken on another workload in minutes. We also do not claim our hardware and model are representative; our results are from one 24 GB accelerator and one 7B model, and extending them is future work.

II. Background and Setup

A. Workload

We replay 200 coding-agent sessions drawn from the TraceLab corpus [1]. Each session is a sequence of rounds; a round presents a prompt of median 23,187 tokens (p90 51,064; maximum 89,436, which alone is 88% of the KV pool), and between rounds the session waits on a tool before returning with a longer context. Arrivals are Poisson at $\lambda = 0.04\,\mathrm{s}^{-1}$ over an 80-minute window, giving 999 steps in total. Two sessions of maximum size cannot be co-resident: their combined contexts exceed the pool by 76%.

The tool wait is the load's defining property and the one most easily misread. Its *median* is 0.35 s, which invites the conclusion that sessions never leave the engine. Its *mean* is 16.85 s, 22% of waits exceed ten seconds, and the maximum is 300 s. We verified that our replay harness honors the trace faithfully: across 799 round transitions the measured gap is 16.85 s mean against a demanded 15.82 s. On a typical transition the harness adds nothing—the median of the per-transition measured/demanded ratios is 1.001—while the ratio of the two means is 1.065, the difference sitting in the tail. The corpus's own characterization makes the same point qualitatively—it reports "diverse and heavily-tailed tool calls" and calls these gaps "human-paced"—but gives no numeric mean [1], so the 16.85 s above is our measurement over the released trace, not a figure we quote from that paper.

B. Engine and the binding constraint

We use SGLang v0.5.10 [2] with Qwen2.5-Coder-7B-Instruct [3] extended to 96K context, on one RTX 3090 (24 GB). KV is paged [4]; at `mem-fraction-static` 0.85 the pool is 101,432 tokens. Hierarchical caching (HiCache), with a host tier of twice the device pool, is enabled on the configurations marked *+HC* and on every Foyer run; the five plain static caps run without it. We keep both because the

host tier turns out to change the mechanism (§III). Every run is guarded: if the pool does not come out at $101{,}432 \pm 20$—which happens when another tenant holds memory on the card—the run is aborted rather than recorded, because SGLang sizes the pool from free memory at startup and a shrunken pool silently produces incomparable numbers.

The relevant consequence is that the pool holds only about four median sessions. Concurrency is therefore capacity-bound, not scheduler-bound, and this is the premise on which everything below rests.

C. The controller under test

Foyer is an admission controller that reads engine telemetry and issues *named* permits rather than a scalar cap: a session is admitted by appearing in a permit file, and the runner refuses to start a round for a session that is not listed. This design choice matters for the comparison, because it is what allows per-session actions at all—including the pause primitive we exploit in §IV-F. It also means that a session, once admitted, holds its permit for its entire lifetime, so the controller never frees capacity by discarding a session's state—it admits fewer sessions instead. That is a statement about the *policy*, not about the memory: the engine's radix cache still evicts parked prefixes under pressure, and we measure that it does and that they are recomputed (§III). We return to this distinction in §VII, because it bounds how far our negative result generalises.

Each control cycle (2 s) the controller computes

$$\Pi \;=\; \max(\rho, \textstyle\sum_i c_i) \;+\; \sum_i g_i, \tag{1}$$

where ρ is the engine's reported occupancy, c_i the context length of live session i, and g_i its forecast growth over the next `horizon` rounds. A session is admitted when $\Pi + n \le B$, with n the candidate's estimated need and B the budget. The three terms are respectively the measured physical occupancy, a reservation for sessions currently parked in a tool wait (whose KV is evictable and therefore absent from ρ), and a reservation for future growth.

D. Baselines and the verdict criterion

Baselines are static caps: at most N sessions resident, $N \in \{1, \ldots, 5\}$, with and without HiCache. We compare on two axes: end-to-end wall clock (time from the first submission to the last completion) and SLO attainment (fraction of requests whose time-to-first-token is under 10 s).

The verdict criterion deserves care, because "faster than cap-N" is not the question. The question is whether a policy reaches a point *above* the static front. A point that lies merely between two static caps is not a win: mixing cap-N and cap-M by time slice approximates any point on the line between them. We flag this segment argument as an argument, not a measurement—we did not run the time-sliced policies, and carrying queue state and cache contents across a switch could bend the line. It is the criterion we apply, and it is conservative in the direction that matters: it can only make our negative result harder to claim, never easier. So we take the upper-left envelope of the static points and report each controller's signed distance above or below it. Every verdict in this paper is produced by one script from the raw logs, and the script's own first version was wrong—it extrapolated past the end of the envelope and declared the slowest static cap a winner. We fixed it and report the episode because it is the failure mode this criterion is most prone to.

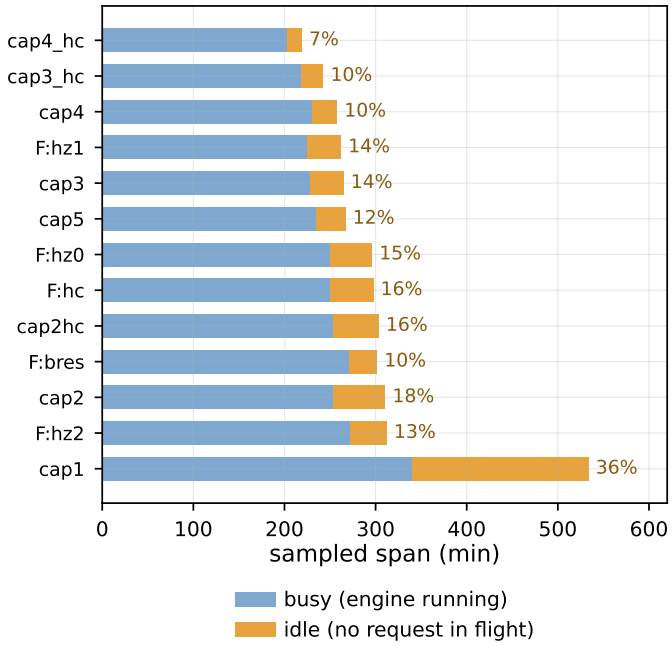


Fig. 2. Wall clock split into busy and idle for every completed run, ordered by total duration. Idle is 7.4% for the fastest configuration and 36.3% for the slowest. Note `bres`: it cuts idle to 10.1% and is still slower than its own baseline, because busy time grows instead.

III. Decomposing the Wall Clock

A. Two components

We split the wall clock at every sampled instant according to whether the engine has any request in flight:

$$W \;=\; W_{\text{busy}} \;+\; W_{\text{idle}}. \tag{2}$$

Fig. 2 shows the split for every completed run. Two things are immediately visible and both matter later.

First, idle time is not noise. It ranges from 7.4% of the wall clock for the fastest configuration to 36.3% for the slowest. The stretches are long: over 1,849 of them the median is 5.6 s, the 90th percentile is 30.1 s, 17.6% exceed 20 s and the longest is 511 s (Fig. 5). A median near the 5 s sampling floor is expected—most tool calls are quick— but the tail is what matters for a scheduler, and it sits at the scale of the 16.85 s mean tool wait.

Second, the two components move independently. `bres` (§IV) reduces idle from 15.9% to 10.1%, recovering about 17 minutes—and is nonetheless *slower* than its own baseline, because its busy time grows by 21 minutes. Any account that treats "more concurrency" as one thing will get this wrong.

B. What determines busy time

During busy time the engine executes steps; each step produces one token per running request. Hence

$$W_{\text{busy}} \approx \frac{G}{\overline{R}}\tau, \tag{3}$$

with G the total generated tokens (fixed by the trace at 706,010) and τ the per-step cost. Across our runs τ is not a fit parameter we tuned; we obtain it two ways and check that they agree.

a) Method 1 (derived).: $\tau_1 = W_{\text{busy}} \cdot \overline{R}/G$. Two choices in that expression are easy to get wrong and both matter. The time term must be the *busy* component, not the wall clock—using the wall clock charges idle time to the steps. And $\overline{R}$ must be the mean running count *during busy time*, not the mean over the whole run: the whole-run mean includes the idle samples, where the count is zero, so it equals $\overline{R}_{\text{busy}}\times$ (busy fraction). Using it makes τ_1 too low by exactly the busy fraction—26% for `cap1`, whose idle share is 36.3%. We report both columns in the data, and use the busy-conditional mean throughout; the whole-run mean is what an earlier revision of this paper used, and it made the two methods agree far better than they do.

b) Method 2 (independent).: For each step record we compute the per-request decode interval, $(\text{total_duration} - \text{first_token})/(\text{output_len} - 1)$, and take the median over steps with $\text{output_len} \geq 32$.

The two do not agree exactly, and the disagreement is informative rather than embarrassing. τ_1 exceeds τ_2 on every run, by 8–40%, and the gap is the share of engine steps that are prefill and therefore emit no output token: τ_1 averages over all steps, τ_2 measures only the decode interval. That share comes out at 15–28% across the five device-only configurations, which is the right order of magnitude for a workload whose prompts are 23,k tokens median against sub-1,k outputs. So the two are not two estimates of one number; they are measurements of two different things, and we now say so. Method 2 remains the one that does not depend on the quantity it is used to explain, and it is the one we quote when the question is the cost of a decode step. The engine's own latency counters (`itl_p50`, `e2e_p50`, `ttft_p50`) are exported as zero by SGLang v0.5.10 in our configuration, so no directly measured step time exists in our data; instrumenting this is future work.

C. The per-step cost is not constant in concurrency

Equation 3 would be uninteresting if τ were a constant. It is not. Across our 13 runs τ_1 ranges from 27.0 to 36.7 ms and τ_2 from 20.6 to 33.1 ms, and both rise with the running count: fitting Eq. 4 to the five device-only configurations gives $\varepsilon = 0.38$ for τ_1 and 0.61 for τ_2 ($R^2 = 0.89$ for the latter, Fig. 4).

The relation is a trend, not a law. Two host-tier configurations with similar running counts behave differently: `cap4_hc` runs 1.73 requests at 29.8 ms while `F:hz0` runs 1.66 at 35.7 ms. Running count sets a floor and a trend, not a value—and on the host-tier family, as we show below, it does not set the value at all.

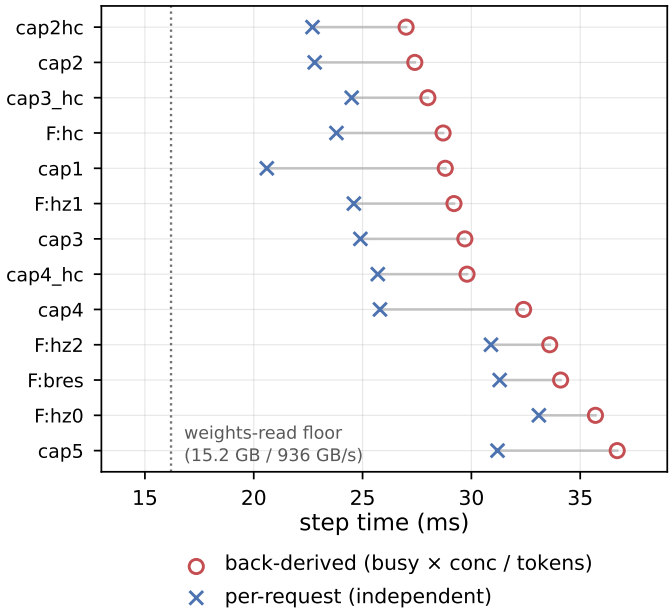


Fig. 3. Per-step cost by two independent measurements, one run per row: circles derive it from busy time, crosses from per-request decode intervals. They agree to within 12% and rank the runs nearly identically ($\rho = 0.967$). The dotted line is the cost of reading the weights once; every run sits above it.

This is the crux of the paper, and we state it at the strength the arithmetic supports. Hold τ_1 at its value for `cap1`, 28.8 ms, and busy time falls as $1/\overline{R}$: raising the running count from 1.00 to 1.84 should have cut busy time by a factor of 1.84, from 340 minutes to 184. The measured curve flattens and lands at 234. The rise in τ takes back 50 of the 156 minutes that the extra concurrency would otherwise have bought—*a third* of the gain, not all of it and, as an earlier revision of this paper claimed, not more than half. The other two thirds are real: raising concurrency does make the engine faster. What it does not do is make it faster in proportion, and the shortfall compounds with every controller that tries to exploit the difference.

a) What the engine is doing when it is not decoding.: SGLang exposes two counters that answer this directly: tokens evicted from the radix cache, and prompt tokens that had to be prefilled rather than reused. Across the five static caps *without* HiCache the two track each other almost exactly. `cap1` evicts 9.2 M tokens and prefills 8.6 M uncached; `cap5` evicts 21.3 M and prefills 20.7 M; the gap is 0.6–0.7 M on every one of the five. An evicted prefix is recomputed, close to one token for one, and that recomputation is chunked into decode steps. Those same five runs give a monotone relation between recomputation and τ: 8.6 M uncached tokens at 18.4 ms, 9.2 at 22.4, 11.5 at 25.5, 15.4 at 29.0, 20.7 at 32.2. That is the mechanism, and on the device-only configurations it explains the τ trend completely.

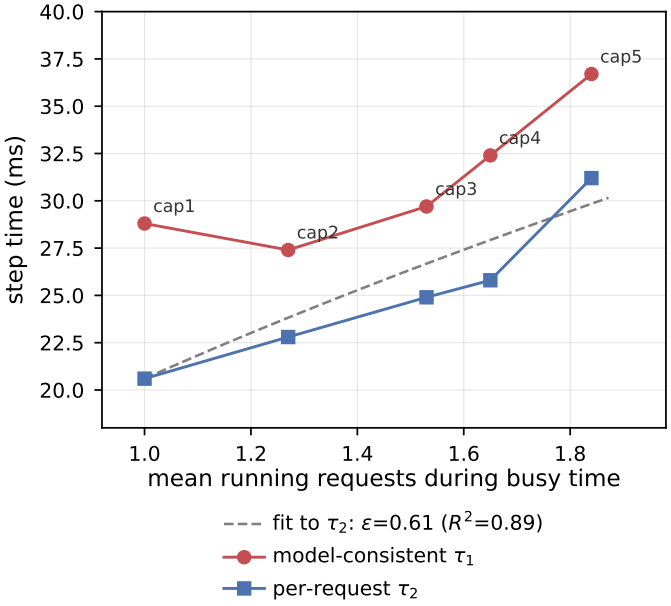


Fig. 4. Per-step cost against the running count during busy time, for the five device-only configurations. Two measurements: τ_1 is model-consistent (it averages over all engine steps, including prefill that emits no output token), τ_2 is the per-request decode interval. The fitted exponent is the elasticity ε of Eq. 4, and τ_1 exceeds τ_2 by 8–40%—that gap is the prefill share.

b) Where the mechanism stops.: With HiCache enabled the correspondence breaks—and it breaks in a way that confirms the mechanism rather than undermining it. The device cache misses as much as before: `cap4_hc` misses 22.4 M tokens on the device, more than the device-only `cap4` misses. But only 9.1 M of that is recomputed; the other 13.3 M is served from the host tier. Across the eight host-tier runs the host serves 13–19 M tokens each, and across the five device-only runs it serves *exactly zero*—the device miss equals the recomputed count to within rounding on all five. Recomputation is still what the device cannot absorb; the host tier is a second absorber in front of it.

That does not resolve the Foyer family, and we say so plainly. Those runs sit at a flat 8.6–8.7 M recomputed tokens while τ ranges from 22.5 to 30.6 ms. Whatever moves τ there is not recomputation, and we do not know what it is. We flag it as the largest open question in the paper, and note that the negative result does not rest on it: the cancellation is measured directly, not inferred from the mechanism.

c) A refutation we had to withdraw.: An earlier revision withdrew this mechanism outright, on two checks that were both wrong. The first was that total prefilled tokens equalled the trace's planned prefill to a ratio of 1.00, so no token could have been computed twice. `prompt_tokens_total` counts the full submitted prompt *including* its cached portion, so it equals the sum of prompt lengths by construction and cannot detect recomputation; the quantity that can is the uncached count used above, which varies by 2.4× across these runs. The second was that SGLang reported no retractions. Radix-cache eviction is not a retraction—retractions count

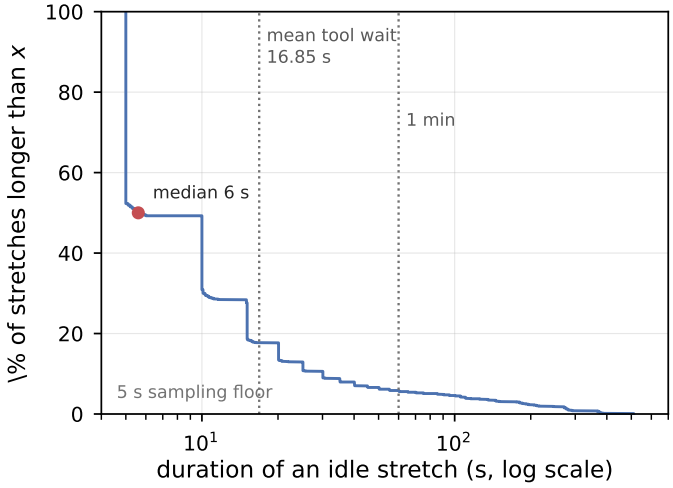


Fig. 5. Survival function of idle-stretch duration, pooled over all completed runs (1,849 stretches). Most stretches are short—the median is 5.6 s—but the minutes are not: 69.6% of all idle *time* sits in stretches longer than 20 s, and the mean stretch is 20.6 s, the same order as the 16.85 s mean tool wait. The cliff is the 5 s sampling floor.

running requests preempted mid-flight, a different event with its own counter. We record the episode because it is the same failure mode as the segment argument of §II: a check that looks like a measurement but is an identity.

D. Why idle time exists, and why recovering it does not pay

Idle time is a property of the workload, not of the controller. A session in a tool wait holds its permit and its KV but issues no request; with only about two live sessions, the probability that all of them are simultaneously waiting is substantial. The direct evidence is the duration of the stretches themselves: they are of the same order as the tool waits, and an order of magnitude longer than the 2 s control cycle (Fig. 5). A controller cannot be blamed for a gap that is set by the tool.

We note one claim we had to drop here. An earlier draft reported that idle fraction falls monotonically with mean concurrency. With the full data it does not: `F:hz0` at 1.41 running requests idles 15.2% while `cap4` at 1.48 idles 10.4%. The weaker statement that survives is the one we use—the stretches are structural and long.

The obvious remedy—keep more sessions live so that someone is always ready—is what `bres` does. It works in the narrow sense: `bres` recovers about 17 of the minutes that its own baseline spends idle. It is still slower overall, because the sessions it keeps live raise the step cost by more than the recovered idle is worth. Idle is not unrecoverable; it is recoverable at a price that is not worth paying here. The structural remedy would be to make the engine do work during a wait, but there is no work to do: the session's next round does not exist until its tool returns.

IV. Results: Four Routes, One Outcome

A. The frontier

Fig. 1 places every completed run on the (wall clock, SLO) plane and draws the upper-left envelope of the static points. Table I gives the numbers and each controller's signed distance from that envelope.

The result is unambiguous. Of nine controller variants, none is above the envelope. The best, a growth-reserve setting of one round instead of the default three, lands $+0.20$ percentage points above the line—and we measured how large a margin means nothing. Two configurations were run more than once: `cap2`+HiCache twice and the Foyer base configuration three times. Their wall clocks have coefficients of variation of 1.6% and 1.8% (ranges 3.2% and 4.0%), and their SLO has standard deviations of 0.5 pp and 0.2 pp. A margin of 0.20 pp is therefore at the replicate spread of the very configuration this paper is about. We record it as *on* the line rather than above it, and we note that we would have needed more than two or three repeats per configuration to resolve a margin this small—which we did not run.

Two features of the front are worth noting before the negative results. The envelope is steep between 213 and 236 minutes (0.76 pp per minute) and shallow thereafter (0.18 pp per minute), so the cost of buying SLO by retreating from the fast end is high. And W is not monotone in concurrency: five sessions (260.9 min) is *slower* than four (213.0 min), because the per-step cost rises faster than the request count does. We return to this non-monotonicity in §III, where it turns out to be the first appearance of the mechanism that drives the whole frontier.

TABLE I

All completed runs. *Conc.* is the time-weighted mean number of *live* sessions, which is not the running-request count $\overline{R}$ used in §III—a live session is parked in a tool wait most of the time, so `cap5` shows 4.89 live against 1.84 running. Δ is the signed distance from the static envelope at that wall clock; positive would mean a genuine win. Three controller runs stopped 1–4 steps short of 999 (`hz0` 995, `hz2` 997, `rg4` 998); they are listed with their measured values and not treated as full runs.

Policy	Wall (min)	Conc.	SLO (%)	Δ (pp)
Static caps				
cap1	527.4	1.00	94.8	—
cap2	304.2	1.99	89.5	−2.2
cap3 (fixed)	258.7	2.89	74.4	−10.7
cap4	249.9	3.94	55.7	−27.9
cap5	260.9	4.89	27.1	−58.4
Static caps + HiCache				
cap2+HC	295.0	1.99	91.6	*anchor*
cap3+HC	236.4	2.93	81.1	*anchor*
cap4+HC	213.0	3.93	63.4	*anchor*
Foyer controller variants				
horizon=1	255.8	2.58	84.8	+0.20
horizon=3 (base)	291.3	2.06	89.8	−1.14
horizon=2	306.1	2.22	86.9	−4.85
resident base	295.5	2.40	83.8	−7.82
horizon=0	290.0	2.69	77.2	−13.5
horizon=1, no hw	310.9	2.77	72.3	−19.6
no gate, no resv.	272.3	3.25	66.6	−20.9
running gate=4	282.3	3.03	64.3	−25.0
all throttles off	415.0	2.58	46.1	−47.2
Independent adaptive controller (§IV-E)				
AIMD, retuned	264.6	3.87	52.8	−33.4
AIMD, paper params	364.3	1.91	81.4	−11.2

B. Route 1: the accounting base

Our controller's projection (Eq. 1) charges a session for its full context even while it is parked in a tool wait, when its KV is evictable and therefore absent from the engine's reported occupancy. We argued this was a double charge and removed it, admitting on engine telemetry alone.

The charge is not double. Removing it did admit more—engine occupancy rose from 41.0% to 52.2% of the pool—and the round-1+ prefix hit rate held at 80.8% against 80.6%, so no eviction spiral occurred. But the run was 4.2 minutes *slower* and lost 6.0 pp of SLO. Per §III, the extra occupancy raised both $\overline{R}$ and τ (31.3 ms against 23.8 ms), and the two cancelled.

We note that this claim had already been falsified once by source inspection: SGLang's `token_usage` subtracts evictable blocks, so a parked session's prefix is genuinely not counted, and charging for it reserves capacity that really will be needed back. The end-to-end measurement agrees with the source reading.

C. Route 2: the growth reserve

Sessions grow by roughly 2.5K tokens per round, so the controller reserves capacity for the next `horizon` rounds. We swept the horizon over $\{0, 1, 2, 3\}$ and the reserve is not a free parameter to be minimised: Fig. 6 shows the response is non-monotone. Removing the reserve entirely costs 13.5 pp of SLO, and a reserve of two rounds is 15 minutes *slower* than three. The optimum is at one round, and there it touches the static front and stops.

D. Route 3: the hard ceiling

Lowering the budget fraction is the crudest available knob and behaves as the other routes do: it slides the operating point along the front. We report it only for completeness, since it adds no mechanism.

E. An independent adaptive controller

Every controller above is ours, which invites the objection that the negative result is a statement about one design rather than about adaptive admission. We therefore implemented a second, mechanistically different controller: an AIMD window over an agent-level admission set, following the control law of the congestion-based system we compare against in §VI. The window grows additively while cache occupancy sits below a low watermark, is cut multiplicatively when occupancy exceeds a high watermark *and* the prefix hit rate falls below a threshold, and holds otherwise. Cuts pause the youngest sessions at their next step boundary; growth resumes paused sessions first. Every input is a runtime gauge—occupancy and hit rate from the engine, nothing from the trace—and the window acts through the same named permits as Foyer.

It does not beat the static front either, at either parameter setting (Table I). At the values reported in that paper's evaluation it finishes in 364.3 minutes at 81.4% SLO, 11.2 pp below the envelope; the same arm with its retuned watermarks finishes in 264.6 minutes at 52.8%, 33.4 pp below. The retuned setting also has the worst segment-to-segment volatility of any run we report (SD 13.2 pp, with segment SLO swinging between 37 and 72), which is what a control loop oscillating around a capacity cliff looks like.

We report two things honestly about this comparison. First, this is our reimplementation of a published control law, not the authors' system, and a reimplementation of any controller can underperform its paper. We built it to the published law, gave it the same permit interface and the same 2 s cycle as Foyer, and did not tune it beyond the two documented settings; but the caveat stands. Second, at its paper parameters it deadlocked on its first attempt—the window pinned at one, no session admitted, the engine idle for over an hour—and we kept the run directory rather than discarding it. A later attempt completed, and that completion is what we report. We read this as a statement about how narrow the stable region is on a pool that holds four sessions, not as a defect of the controller in its intended setting; both settings sit below the front regardless.

F. Route 4: execution granularity

The first three routes all assume that what a controller can set is *how many sessions are resident*. A static cap sets the same quantity, so it is unsurprising that they land in the same place. The fourth route attacks the assumption instead: because a large fraction of live sessions are parked at any instant, a cap on *resident* sessions leaves the engine idle whenever all residents happen to be parked. Capping the *running-request* count instead should keep the engine busy while leaving the resident count high.

The permit design supports this directly: a paused session finishes its current round and blocks before its next one, and resumes when the controller re-admits it. We implemented it as a gate on the engine's own `num_running_reqs` and swept the gate over $\{2, 4\}$.

It does not work, and the reason is structural rather than a tuning matter. Since a live session is running about 45% of the time, the running count is approximately $0.45 \times$ (live count). Constraining the live count to N therefore constrains the running count to $0.45N$, which is below the gate—so the gate can only throttle, never pin. Both repairs fail: without a constraint on admission, a paused session is re-admitted on the next cycle and blocks for at most one 2 s cycle; with one, the policy is a session cap again. The obstacle is granularity—`paused` takes effect at round boundaries, while pinning an instantaneous count requires control the permit mechanism does not have.

The measured outcome matches: the gate drove running concurrency from 1.42 to 1.29 and live sessions from 2.78 to 1.91. It tightened rather than pinned, and a tightening is another cap.

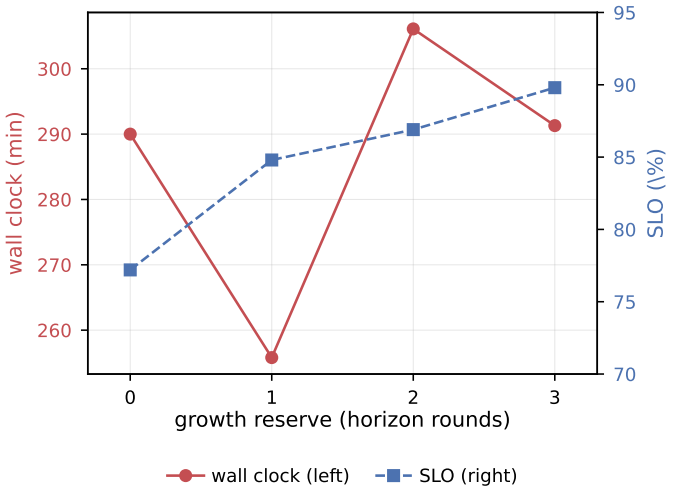


Fig. 6. The growth reserve is not a free parameter. Removing it costs 13.5 pp of SLO; two rounds is 15 minutes slower than three. The optimum at one round touches the static front and does not cross it.

V. Load Dependence

A negative result about adaptivity invites the obvious objection: adaptivity pays when conditions *change*, and a single stationary trace is the one case where a static cap should be near-optimal. We agree, and we test it.

We run the same controller and the same cap family on a 25-session tier. The throttle configuration is reversed: with all safety throttles disabled, the 25-session tier runs *51% faster* (58.4 to 28.8 minutes) at a cost of only 2.4 pp of SLO, while on the 200-session tier disabling the same throttles costs 42% (291.3 to 415.0 minutes) and 43.7 pp of SLO. One switch, two regimes, opposite signs, both around 40–50% (Fig. 7). No single scalar setting is optimal across both.

This is the regime in which adaptive control should win, and we report two findings about it. The first is discouraging for our controller: on the 25-session tier it does not win. A static cap of three, re-run under the current harness, finishes in 24.2 minutes at 91.2% SLO—16% faster than the most aggressive Foyer configuration we measured on that tier, and with better run-to-run stability (segment SD 5.1 against 11.0 for cap-2). The stateless cap, which cannot see occupancy, backlog or remaining work, is simply better here. We flag one caveat and one gap. The caveat is that the tier comparison above is cross-generation: the 25-session throttle arms come from an earlier harness revision and are not directly comparable to the 200-session numbers, so we treat them as a direction rather than a magnitude. The gap is that the matching Foyer arm on the 25-session tier under the current harness was interrupted by another tenant and is not reported.

The second finding is the more useful one. The signal an adaptive controller would need is available—the number of sessions waiting for admission differs by 6–7$\times$ between the tiers (median 9–11 at 25 sessions against 66 at 200, maximum 135)—but its *interpretation* is not obvious. At the 25-session tier the controller holds eleven sessions back while running

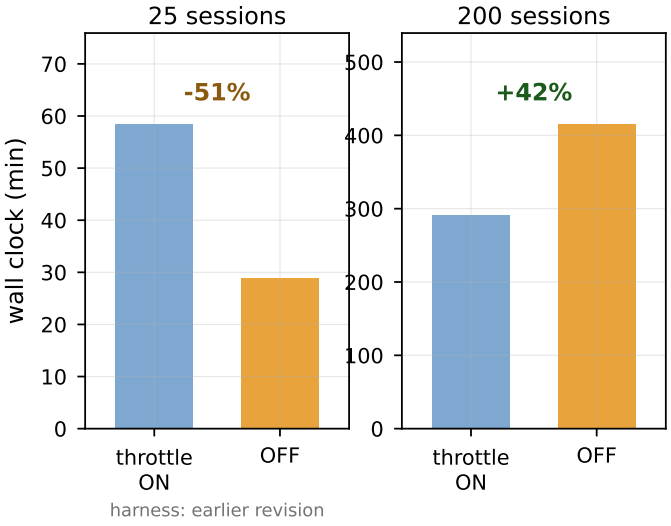


Fig. 7. The same throttle switch, two load tiers, opposite signs. The 25-session bars are from an earlier harness revision and are shown as a direction, not a magnitude; the matching current-harness arm was interrupted and is not reported.

exactly one; that backlog is manufactured by the throttling itself rather than evidence that demand has slackened. Reading it correctly requires combining instantaneous backlog with progress through the run, which is a stateful computation a cap cannot perform. We regard this as the most promising direction we did not finish, and we are explicit that we did not finish it.

VI. Related Work

a) Admission and scheduling for agentic serving.: Recent systems report large gains from treating an agent session, rather than a request, as the schedulable unit, and from preserving session state across tool calls: session-aware admission with CPU-restorable KV [5], [6], program-level pause/resume with shortest-context-first restoration [7], and congestion-based control over an agent-level window [8]. A recurring measurement in this line is that recomputation, not admission, dominates: [8] attributes 49.1% of end-to-end latency to recomputation, and [9] preserves state on the CPU at the cost of a reload. Our results are consistent with this literature rather than in tension with it, and §VII explains why: the mechanism these systems add is one our controller already has.

b) Engine-side techniques.: A separate line improves the engine rather than the admission decision. Iteration-level batching and paged KV [4], [10] are the substrate every policy here runs on; chunked prefill with stall-free batching [11], prefill/decode disaggregation [12], and cache-centric scheduling [13] are the refinements. Our §III attributes 7–10 ms of each 26 ms step to non-decode work, which is exactly what this line attacks. We emphasise that these techniques raise the achievable rate for *all* policies; they move the frontier rather than deciding who sits above it.

c) State retention and preemption.: A third line keeps session state alive across the gaps we measure in §III: keeping KV on cheaper storage across turns [14], [15], migrating it between instances [16], and preempting at iteration granularity [17]. These systems exercise finer control than our 2 s admission cycle, which is the boundary of our negative result: we falsify one control granularity, not the family. What they share with Foyer is that the state is not discarded—which is why, as we argue below, adding admission intelligence on top buys nothing here.

d) Application-level dependencies and replay.: Serving an LLM *application* rather than a request means the scheduler sees dependencies between calls [18], and agentic traces have structure that request-level benchmarks do not capture [19]. Our replay preserves that structure but fixes it in advance, which is exactly the limitation these works would press on.

e) Prefix caching.: The prefix cache is often proposed as the signal an admission policy should use. In our workload it is not a policy variable: round-1+ hit rate equals the trace's own planned value (89.8%) to within one percentage point across four different admission policies, so admission does not move it. We note the boundary of this claim—we varied admission policy, not eviction policy, and SGLang exposes alternate eviction policies we did not test. A characterization of production KV-cache traffic at a large cloud provider finds that eviction-policy design is "highly workload-dependent" and that reuse patterns differ sharply by request category [20]; that is reason to expect a single trace family to understate the space, though it is not a policy comparison and we do not present it as one.

VII. Discussion

a) Why published gains do not reappear.: The systems cited above win by keeping session state alive across tool waits and by pausing at agent granularity rather than request granularity. Foyer has both properties by construction: the permit file gives per-session pause and resume, and the controller never frees capacity by discarding a session—it admits fewer sessions instead. To that extent the mechanism these systems add is one our controller already has, which is one reason adding admission intelligence on top of it buys nothing here.

We are careful not to overstate this, because the distinction is between *choosing* to discard state and *having* it evicted. Retention here is the engine's radix cache doing its ordinary job; it is not something the controller implements, and §III measures that the cache evicts parked prefixes under pressure and that they are recomputed. What Foyer avoids is the deliberate trade of state for capacity. A system that manages the host tier as an explicit policy—as [9] and [14] do—holds a lever we do not, and our negative result does not cover it.

b) A criterion, not just a verdict.: The decomposition turns the question into one number. Throughput during busy time is $\overline{R}/\tau$, so raising concurrency by a factor f buys a factor $f^{1-\epsilon}$, where

$$\epsilon = \frac{\mathrm{d}\ln\tau}{\mathrm{d}\ln\overline{R}} \tag{4}$$

is the elasticity of per-step cost with respect to the running count. At $\epsilon = 0$ the step cost is flat, concurrency converts one for one into throughput, and an admission controller has room to work. At $\epsilon = 1$ the two cancel exactly and it has none. Everything in between is a quantitative statement about how much room, and it is measurable in minutes from per-request logs a serving system already writes.

On the five device-only configurations, where §III establishes that the step cost is driven by recomputation, $\overline{R}_{\text{busy}}$ runs from 1.00 to 1.84. A least-squares fit of Eq. 4 over those five points gives $\epsilon = 0.61$ on the per-request measurement ($R^2 = 0.89$), or 0.38 on the model-consistent one (Fig. 4). The practical reading is that every 10% of added concurrency buys roughly 4–6% of throughput, depending on which step cost you can measure—which is why nine controller variants and an independent AIMD controller all fail to move the frontier, and why the one that came closest landed on it rather than above. We offer this as the paper's constructive output: measure ϵ before building a controller, and build one only where it is small. We give both values rather than choosing, because the choice is the reader's measurement capability, not ours.

c) The threshold decides the verdict, and we report that.: Every SLO number in this paper counts a request as good if its time-to-first-token is under 10 s. That constant sets the whole vertical axis, so we re-derived the frontier at 5 s and 20 s as well, from the same per-request logs. The ordering of the static caps does not change. What changes is where the controllers sit relative to them, and at the loose end one of them crosses:

SLO threshold	5 s	10 s	20 s
`horizon=1`	-0.5	$+0.2$	$+2.7$
Foyer base	-2.1	-1.1	$+0.7$
`bres`	-11.9	-7.8	-4.3
Concur-style AIMD	-14.2	-11.2	-6.5

Two readings, and we give both. The first is that our headline claim is conditional: at the strict threshold that interactive serving actually cares about, no controller reaches the static front; at a loose one, one does. The second is that the crossing is still not a win. $+2.7$ pp at 20 s is larger than the replicate spread of two of the three configurations we repeated (0.5 and 1.0 pp of SLO) but smaller than the third's (12.2 pp), so it is inside the noise we can measure. A margin that appears only at one threshold and only at a size we cannot resolve is not evidence that adaptive admission works; it is evidence that the question needs more runs than we have. We state the conditional version in the abstract, and we flag this as the second thing we would re-run with more GPU time.

d) Limitations.: Our results come from one 24 GB accelerator, one 7B model and one trace family; generality is untested. Only two configurations were repeated, twice and three times respectively, so the replicate spread we report in §IV is estimated from two degrees of freedom at best; a margin as small as the one that decides our verdict would need several more repeats per configuration to resolve, and we did not run them. Two specific boundaries matter. First, the pool holds about four median sessions, so concurrency is capacity-bound by construction; on a device with a much larger pool the premise may not hold and the conclusion may invert—we regard reproducing on a larger device as the single most important next experiment. Second, the load is stationary, single-tenant and single-SLO. The load-dependence result of §V says the reverse should be expected when conditions vary, and multi-tenant or heterogeneous-SLO settings are where we would look next.

e) Reproducibility.: Every verdict in this paper is produced by one script from the raw per-step logs, and every run is guarded against a shrunken KV pool. We report six claims we retracted during the work, because the retractions were the most informative events. In order of how much they cost us:

The denominator of the step cost. Our first §III divided by the whole-run mean running count while multiplying by busy time. The two are not the same quantity—the whole-run mean includes the idle samples—so τ came out low by exactly the busy fraction, 36% for `cap1`, and two independent errors cancelled to make our two measurement methods appear to agree. This is the one that changed a headline number.

A mechanism refutation that was itself refuted: we withdrew the eviction-to-recomputation account on the strength of two checks that were both identities rather than measurements (§III).

A median-based reading of the tool wait that inverted a central assumption. *A monotone relation between idle fraction and concurrency* that the full data does not show. *A growth-sensitivity conclusion* that rested on an arm we had already invalidated. And *the accounting double-charge* of §IV, falsified twice—once by source inspection and once by the end-to-end measurement here.

VIII. Conclusion

We asked whether an adaptive admission controller can beat a tuned static concurrency cap on a memory-bound agentic LLM workload, and found that on our system it cannot: nine variants, four mechanistically distinct routes, none above the static front. The reason is a cancellation. Admission policy controls how many requests are in flight; the per-step cost rises with that number; and the product is what the wall clock sees. We gave a two-part decomposition of the wall clock that makes the cancellation measurable, identified a structural idle component the policy cannot recover, and showed the regime in which adaptivity should have paid—where the same switch is worth $+51\%$ and -42%—is one where a stateless cap still wins. The constructive output is a per-request measurement that decides, in minutes, whether admission control has room on a given workload.

References

[1] K. Zhu, M. Jacob, C. Ma, Y. Pan, S. Wang, A. Krishnamurthy, and B. Kasikci, "TraceLab: Characterizing coding agent workloads for LLM serving," 2026, preprint; this paper's trace source.

[2] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024, pp. 62 557–62 583, arXiv:2312.07104; v1 title was "Efficiently Programming Large Language Models using SGLang".

[3] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu, K. Dang, A. Yang, R. Men, F. Huang, S. Quan, X. Ren, X. Ren, J. Zhou, and J. Lin, "Qwen2.5-coder technical report," 2024.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with PagedAttention," in *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP '23)*, 2023, pp. 611–626.

[5] U. Meng, U. Zhao, D. Li, G. Gao, and F. Du, "Talaria: Session-aware serverless serving of hundred-billion-parameter LLMs," 2026, preprint; author names as listed on arXiv.

[6] H. Li, R. He, Q. Mang, Q. Zhang, H. Mao, X. Chen, H. Zhou, H. Zhang, A. Cheung, J. Gonzalez, and I. Stoica, "Continuum: Efficient and robust multi-turn LLM agent scheduling with KV cache time-to-live," 2025, preprint, no venue.

[7] H. Kang, Z. Li, W. Xu, X. Yang, Y. Chen, J. Wang, B. Chen, T. Krishna, C. Xu, and S. Arora, "ThunderAgent: A simple, fast and program-aware agentic inference system," 2026, preprint, no venue.

[8] Q. Chen, Z. Ye, T. Tang, P. Sun, B. Tian, G. Wang, S. Li, Y. Wen, Z. Han, and T. Zhang, "CONCUR: High-throughput agentic batch inference of LLM via congestion-based concurrency control," in *International Conference on Machine Learning (ICML 2026)*, 2026, poster, Seoul, July 2026; OpenReview id U7c6glPdDW; arXiv:2601.22705.

[9] T. Xia, H. Li, Z. Li, X. Chen, H. Kang, Y. Qiao, Y. Xu, and I. Stoica, "Idleness is relative: Exploiting tool-call idle windows for offloading in agentic systems with MORI," 2026, preprint, no venue.

[10] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*. USENIX Association, 2022, pp. 521–538.

[11] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. Gulavani, A. Tumanov, and R. Ramjee, "Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 117–134.

[12] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 193–210.

[13] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A KVCache-centric disaggregated architecture for LLM serving," *ACM Transactions on Storage*, vol. 22, no. 4, pp. 1–38, 2026.

[14] L. Yu, J. Lin, and J. Li, "Stateful large language model serving with Pensieve," in *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 2025, pp. 144–158.

[15] B. Gao, Z. He, P. Sharma, Q. Kang, D. Jevdjic, J. Deng, X. Yang, Z. Yu, and P. Zuo, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *2024 USENIX Annual Technical Conference (USENIX ATC '24)*. USENIX Association, 2024, pp. 111–126.

[16] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, "Llumnix: Dynamic scheduling for large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 173–191.

[17] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "FastServe: Iteration-level preemptive scheduling for large language model inference," in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '26)*. USENIX Association, 2026, pp. 57–74.

[18] C. Lin, Z. Han, C. Zhang, Y. Yang, F. Yang, C. Chen, and L. Qiu, "Parrot: Efficient serving of LLM-based applications with semantic variable," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, 2024, pp. 929–945.

[19] M. Wang, Y. Yue, S. Li, Y. E. Zhou, C. Wang, and J. Huang, "Benchmarking LLM serving systems for agentic AI workloads with XPerf," 2026, preprint.

[20] J. Wang, J. Han, X. Wei, S. Shen, D. Zhang, C. Fang, R. Chen, W. Yu, and H. Chen, "KVCache cache in the wild: Characterizing and optimizing KVCache cache at a large cloud provider," in *2025 USENIX Annual Technical Conference (USENIX ATC '25)*. USENIX Association, 2025, pp. 465–482.